



暨南大学
JINAN UNIVERSITY

研究生课程论文

论文题目： 语言模型微调实践

课程名称： 自然语言处理

学 院： 智能科学与工程学院/人工智能产业学院

学 系：

专 业： 计算机科学与技术（人工智能方向）专业

高级研修项目

学生姓名： 陈燕松

学 号： 202560204520004

指导教师： 汤知日

2026 年 05 月 31

Practice of Language Model Fine-Tuning

Abstract: This project documents the experience of fine-tuning embedding models and large language models (LLMs), as well as the practical effects of fine-tuning for question-answering tasks. It covers retrieval task practices using bge-m3, LoRA fine-tuning experiments, the construction of RAG (Retrieval-Augmented Generation) systems with LLMs, and attempts at CPT (Continual Pre-Training) and LoRA fine-tuning on Qwen/Qwen3.5-0.8B-Base. The project code is available at: <https://github.com/EthanGD/NLPHomeWork20260530/tree/main>

Keywords: NLP; LLM; CPT; LoRA; SFT; Sentence Similarity

1 Introduction

The purpose of this practice is to hypothesize an ordinary small or medium-sized enterprise user with limited computational resources, and to experiment with various methods for incorporating knowledge not originally present in the model, evaluating the effectiveness of question-answering tasks under different approaches. The current background constraints are threefold: first, limited computational power; second, a relatively small dataset scale; and third, the lack of specific, in-depth domain knowledge in open-source models. The objective of this study is to explore how, under these constraints, local models can be extended with specific, deep domain knowledge to improve domain-specific question-answering performance. Due to limited computational resources and dataset size, full training of LLMs is infeasible; therefore, only fine-tuning methods are considered in this research. Ultimately, we selected LoRA and RAG enhancement methods for initial experiments, as they require minimal computational power and data scale. Recognizing the importance of enabling the LLM itself to understand

domain knowledge, we later incorporated the Continued Pre-Training (CPT) method as well. All completed experiments have been uploaded to GitHub, and the code for each experimental project is described as follows:

Project code v1 includes the following modules:

(1) Utilizing bge-m3 to vectorize question-answer pairs, storing them in a database with support for addition, deletion, and similarity-based retrieval, effectively serving as a knowledge base management module.

(2) LoRA fine-tuning of bge-m3.

(3) Constructing a RAG-paradigm Agent to compare the effects of fine-tuning bge-m3.

Project code v2 tested the generation performance of the original Qwen3.5-0.8B model. It also fine-tuned Qwen3.5-0.8B using LoRA with 208 question-answer pairs about OpenClaw, and tested the generation performance of the fine-tuned model.

Project code v3 utilized text content from six traditional Chinese medicine acupuncture books to fine-tune Qwen3.5-0.8B using CPT (Continual Pre-Training), and tested the generation performance of both the original and fine-tuned models.

Project code v4 used text content from six traditional Chinese medicine acupuncture books to generate question-answer pairs via Qwen3.5-122B. It then fine-tuned Qwen3.5-0.8B using LoRA and evaluated the results.

The main contributions of this paper include:

(1) A systematic comparison of four domain adaptation methods, embedding model fine-tuning, LoRA generative fine-tuning, continued pre-training, and LoRA+SFT, under identical computational resource constraints, providing a reproducible benchmark.

(2) An in-depth analysis of the manifestation and causes of catastrophic forgetting during continued pre-training, along with validation of SFT as an effective recovery strategy.

(3) An exploration of the feasibility of using large-model-generated data to assist small-model fine-tuning, offering a low-cost solution for SMEs facing data scarcity.

2 Experimental Environment

Single NVIDIA L20 GPU, Python 3.11.

3 Data Preparation

3.1 Data Source 1

OpenClaw question-answer pairs were crawled from the web (<https://docs.openclaw.ai/zh-CN/help/faq>, <https://fetchopenclaws.com/answers>, <https://www.meta-intelligence.tech/insight-openclaw-faq>).

3.2 Data Source 2

A traditional Chinese medicine acupuncture text dataset was downloaded from HuggingFace (<https://huggingface.co/datasets/wobure/zhongyi-books>).

4 v1: bge-m3

4.1 Experimental Objective

Using the prepared question-answer pairs, we performed LoRA fine-tuning on the embedding model BAAI/bge-m3 and observed the results. Simultaneously, we constructed a RAG-paradigm Agent to compare the differences among RAG generation, pure LLM generation, and pure word-vector retrieval.

4.2 Data and Preprocessing

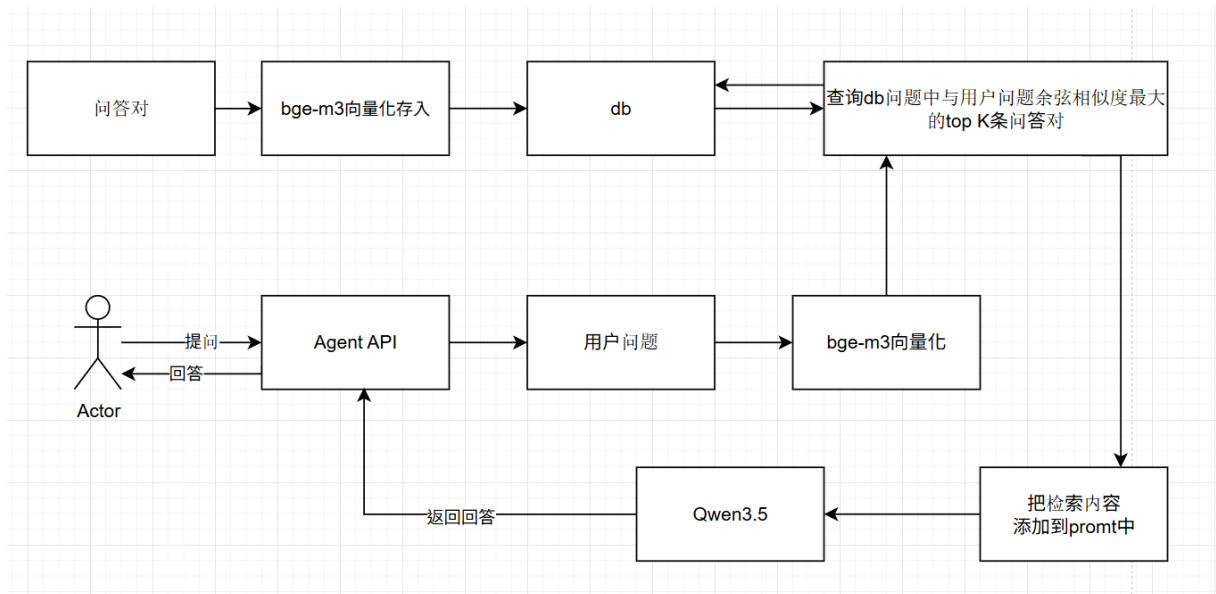
The data source consists of 208 OpenClaw question-answer pairs (Query-Positive pairs) crawled from the web as mentioned in Section 3.1. The dataset was split into a training set (166 samples) and a test set (42 samples) at an 80/20 ratio. Each sample is formatted as an anchor

(query) and a positive (correct answer), with negative samples dynamically sampled during training.

4.3 Embedding Model LoRA Training Configuration

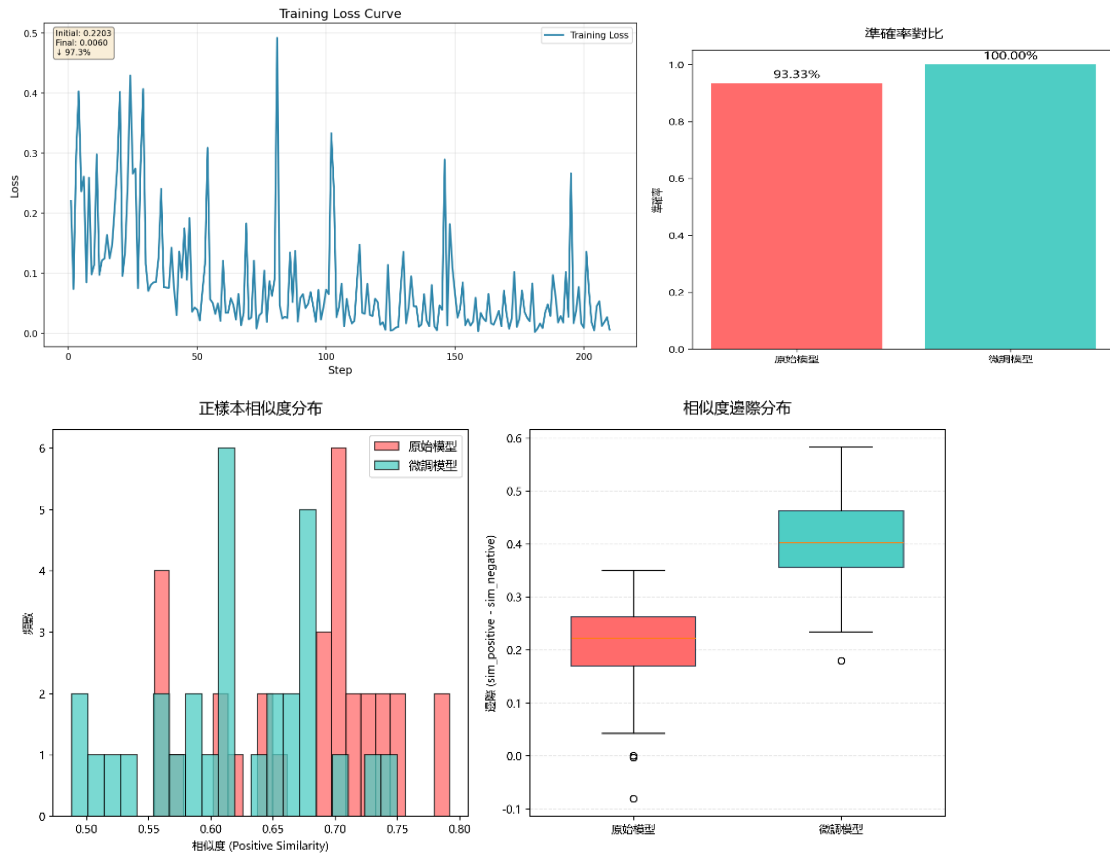
Configuration	Value
Base Model	BAAI/bge-m3 (SentenceTransformer framework)
Loss Function	MultipleNegativesRankingLoss
Training Epochs	10 epochs
Batch Size	4×2 (gradient accumulation) = 8
Learning Rate	2e-5
Precision	bf16
Optimizer	AdamW
Evaluation Strategy	Per-epoch evaluation

4.4 RAG Architecture



This experiment constructed a RAG (Retrieval-Augmented Generation) system that combines vector retrieval with LLM generation. The bge-m3 model performs semantic retrieval, and the retrieved results are fed as context into the LLM for answer generation.

4.5 Experimental Results



The training loss decreased from an initial 0.182 to a final 0.020, representing an 89% reduction, indicating good model convergence. The evaluation results are as follows:

Metric	Training Set	Test Set	Remark
Accuracy	100%	100%	Unchanged
Positive Similarity	0.623	0.600	Slight decrease
Negative Similarity	0.266	0.270	Stable
Average Margin	0.357	0.330	—

Compared to the original model, the fine-tuned model improved negative sample discrimination by 48.8% and the average margin by 66.4%.

4.6 Analysis

The v1 experiment demonstrates that fine-tuning the embedding model on question-answer pair data can significantly improve retrieval discrimination. The margin improvement from 0.238 to 0.396 (+66.4%) means the model can more clearly distinguish between relevant and irrelevant documents. The slight decrease in positive sample similarity ($0.677 \rightarrow 0.621$) suggests that more domain-specific positive samples are needed to maintain recall. Using `test_list_api.py`, the retrieval performance showed no significant change. In the future, more cases could be added to enrich the data for further experimentation. Overall, the v1 fine-tuning was successful and is suitable for the retrieval component of RAG systems.

Additionally, RAG provides significant enhancement for content that the LLM lacks. When asked questions closely related to the knowledge base, the answer accuracy reaches 100%. However, only 4 test cases were prepared in this experiment, and further validation logic needs to be enriched.

5 v2: Qwen3.5 LoRA Fine-Tuning

5.1 Experimental Objective

The objective of v2 shifted from embedding model fine-tuning to post-training. Using the Qwen3.5-0.8B multimodal large language model, we employed LoRA parameter-efficient fine-tuning to achieve answer generation for knowledge about OpenClaw that the model does not inherently possess.

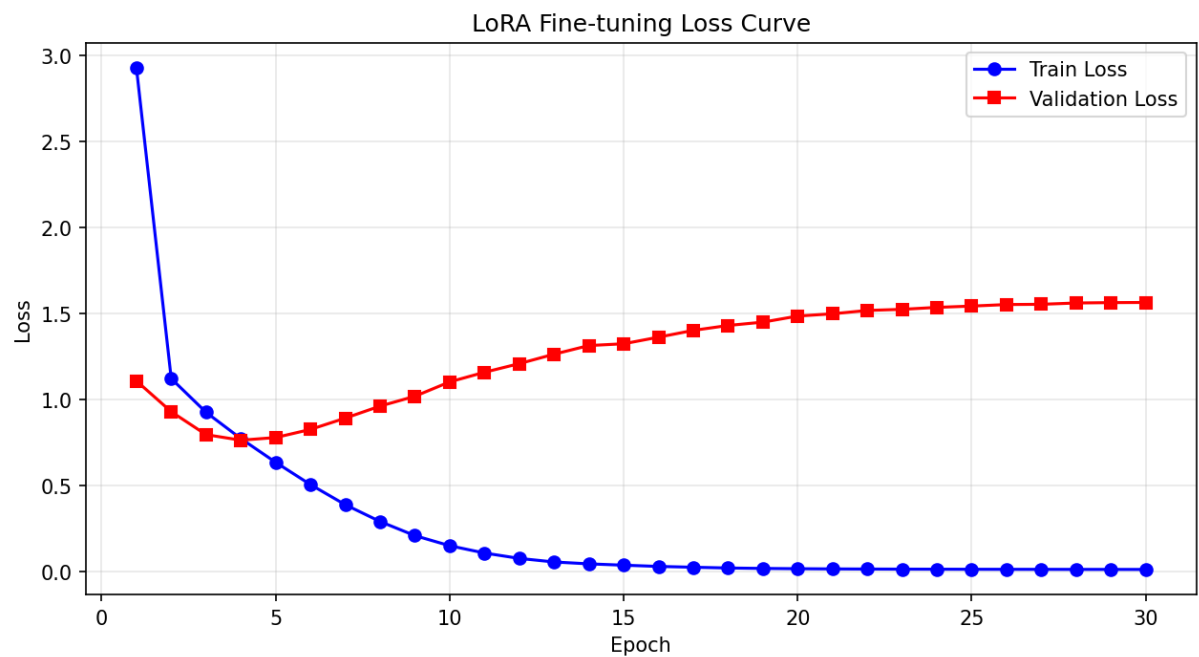
5.2 Data and Preprocessing

The data source consists of 208 OpenClaw question-answer pairs (Query-Positive pairs) crawled from the web as mentioned in Section 3.1. The dataset was split into a training set (175 samples) and a validation set (32 samples) at an 85/15 ratio. The data was formatted using the text template: "Question: {query}\n\nAnswer: {pos}".

5.3 LoRA Training Configuration

Configuration	Value
Base Model	Qwen3.5ForConditionalGeneration (0.75B parameters)
Fine-tuning Method	LoRA (r=8, alpha=16, dropout=0.05)
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Trainable Parameters	$3,194,880 / 856,180,800 = 0.37\%$
Training Epochs	30 epochs
Batch Size	8
Learning Rate	$2e-4$
Optimizer	AdamW
LR Schedule	Linear warmup (10% steps) followed by linear decay

5.4 Experimental Results



Training was completed in under 10 minutes. The training process exhibited a typical overfitting pattern:

Metric	Initial	Final	Trend
Train Loss	2.93	0.015	Continuously decreasing
Val Loss	1.11	0.764 (best at epoch 4) → 1.65+	Decrease then increase
Best Model	—	Epoch 4 (val loss = 0.764)	—

The generation evaluation results were unsatisfactory: the model produced a large amount of repetitive output and exhibited underfitting behavior.

5.5 Analysis

The key issue in v2 is severe overfitting. The train loss continuously decreased to near zero, while the validation loss began to rise after 4 epochs, which is a classic overfitting signal. The main reasons are: (1) the data volume is too small (only 175 training samples); (2) the number of epochs is too high (30); (3) the validation set is too small (32 samples), leading to unstable evaluation. The generated answers exhibited repetitive loops or irrelevant responses, indicating that the model did not truly learn the QA mapping but rather mechanically memorized the training data.

6 v3: Qwen3.5 Continual Pre-Training (TCM Domain Injection)

6.1 Experimental Objective

We sought data from the traditional Chinese medicine (TCM) acupuncture domain and performed continual pre-training on the Qwen3.5-0.8B-Base model to evaluate the post-training model performance.

6.2 Data and Preprocessing

The data source consists of 6 classic TCM acupuncture book texts in Markdown format from HuggingFace, totaling 452,466 characters. The collection covers "Huangdi Neijing · Suwen", "Huangdi Neijing · Lingshu", "Zhenjiu Jiayi Jing", "Jingluo" (Meridians), "Shuxue" (Acupoints), and "Shi Er Jingmai" (Twelve Meridians).

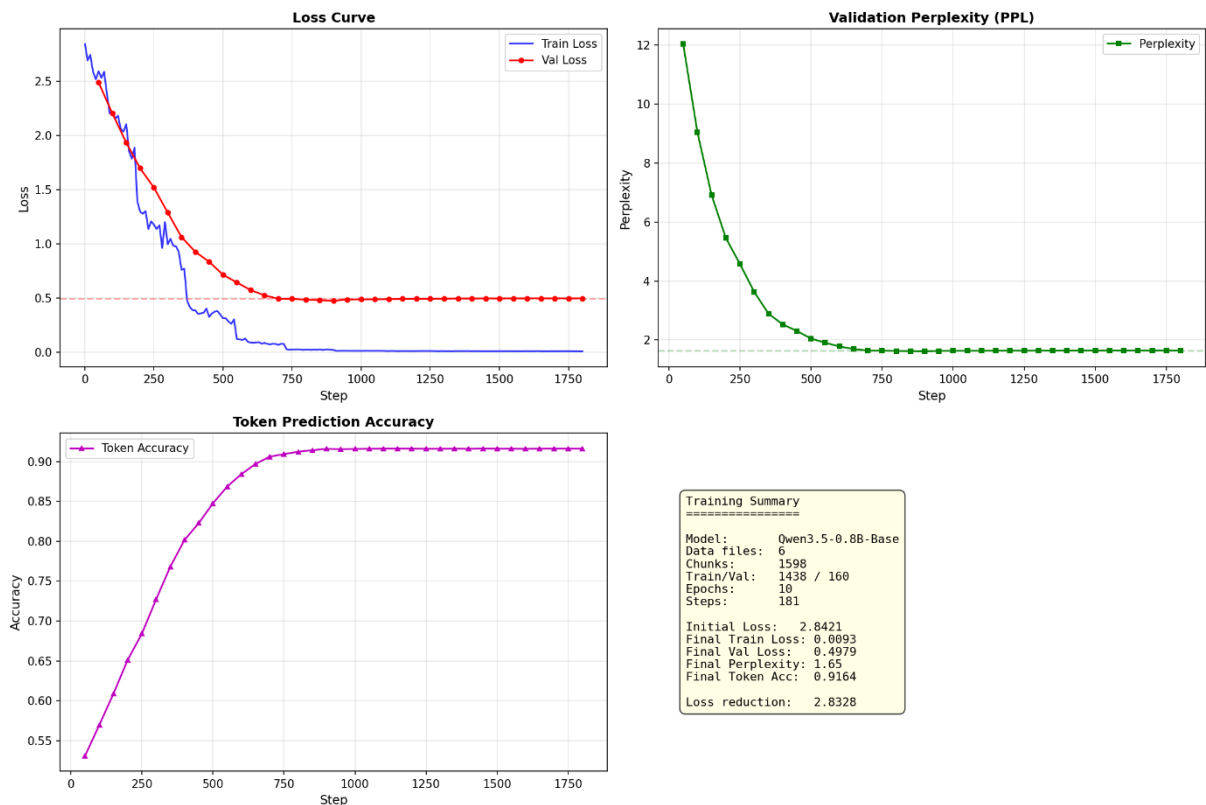
Preprocessing pipeline: remove Markdown markers (#, *) → plain text → tokenization → chunking. Chunking strategy: max_length=512 tokens, stride=256 (overlap), generating a total of 1,598 chunks. Training/validation split: 1,438 / 160.

6.3 Training Method and Core Characteristics

v3 adopts the Continual Pre-Training (CPT) paradigm, whose core characteristic is labels = input (pure text continuation task). This is fundamentally different from SFT: the model learns to predict the next token rather than learning question-answer mappings. Training used full-parameter fine-tuning (not LoRA), with batch size = 1, gradient accumulation over 8 steps (effective batch size = 8). Learning rate = $5e-5$, trained for 10 epochs.

6.4 Experimental Results

Qwen3.5-0.8B-Base Continual Pre-training — Training Metrics



Training took 4 hours and 10 minutes, making it the most time-consuming experiment among all versions.

Metric	Initial	Final
Train Loss	2.84	≈ 0.009

Val Loss	2.52	≈ 1.75 (continuing downward trend)
Perplexity	≈ 12.4	≈ 5.75
Token Accuracy	57.6%	91.65%

6.5 Comparative Test Results and Analysis

Comparative testing was conducted using a question-answering approach on both the original model and the fine-tuned model, revealing critical issues:

Test Question	Original Model	CPT Fine-tuned Model
Which acupoints can be needed for abdominal distension?	Listed acupoints after reasoning	Continued writing classical text from "Zhenjiu Jiayi Jing"
Which acupoints can be needed for headache?	Analyzed headache types and gave suggestions	Continued writing passages from TCM classics
What are the twelve meridians?	Correctly listed the meridians	Continued writing text from "Huangdi Neijing · Lingshu"
What are the effects of Zusanli?	Provided detailed efficacy description	Listed functions and combinations in classical format

Diagnosis of training methodology:

(1) Mismatched task objectives: The goal of CPT is text continuation (Next Token Prediction), not question answering. The original model is a general-purpose QA model, while the CPT-trained model learned the pattern of continuing from classics.

(2) Catastrophic Forgetting: To learn the linguistic patterns of TCM texts (Classical Chinese style, classical citation structure), the model significantly weakened its original general QA capabilities. General knowledge (language understanding, commonsense reasoning) and QA abilities were disrupted, while TCM domain knowledge was enhanced.

(3) Evaluation mismatch: Using a QA approach to evaluate a continuation model is essentially an error in the evaluation dimension. The success of CPT is reflected in loss reduction and token accuracy improvement, but these metrics do not directly correspond to QA capability.

7 v4: Qwen3.5 LoRA

7.1 Experimental Objective

We used an LLM to automatically generate question-answer pairs based on the text content of classic TCM acupuncture books. We then fine-tuned Qwen3.5-0.8B using LoRA with these generated QA pairs, observing the quality of the generated QA pairs and the response performance of the resulting fine-tuned model.

7.2 Data Construction

The data for project v4 also originates from the six classic TCM acupuncture books on HuggingFace. The texts were first segmented into chunks, and each chunk was injected into a prompt to generate question-answer pairs.

The chunking logic is as follows:

A theoretical chunk length of 3,000 characters is used to read a temporary chunk. Within this temporary chunk, the position of the last sentence-ending punctuation mark is identified, thereby producing the actual chunk used.

The generated QA pairs are saved in `qa_dataset.jsonl`. An example entry is as follows:

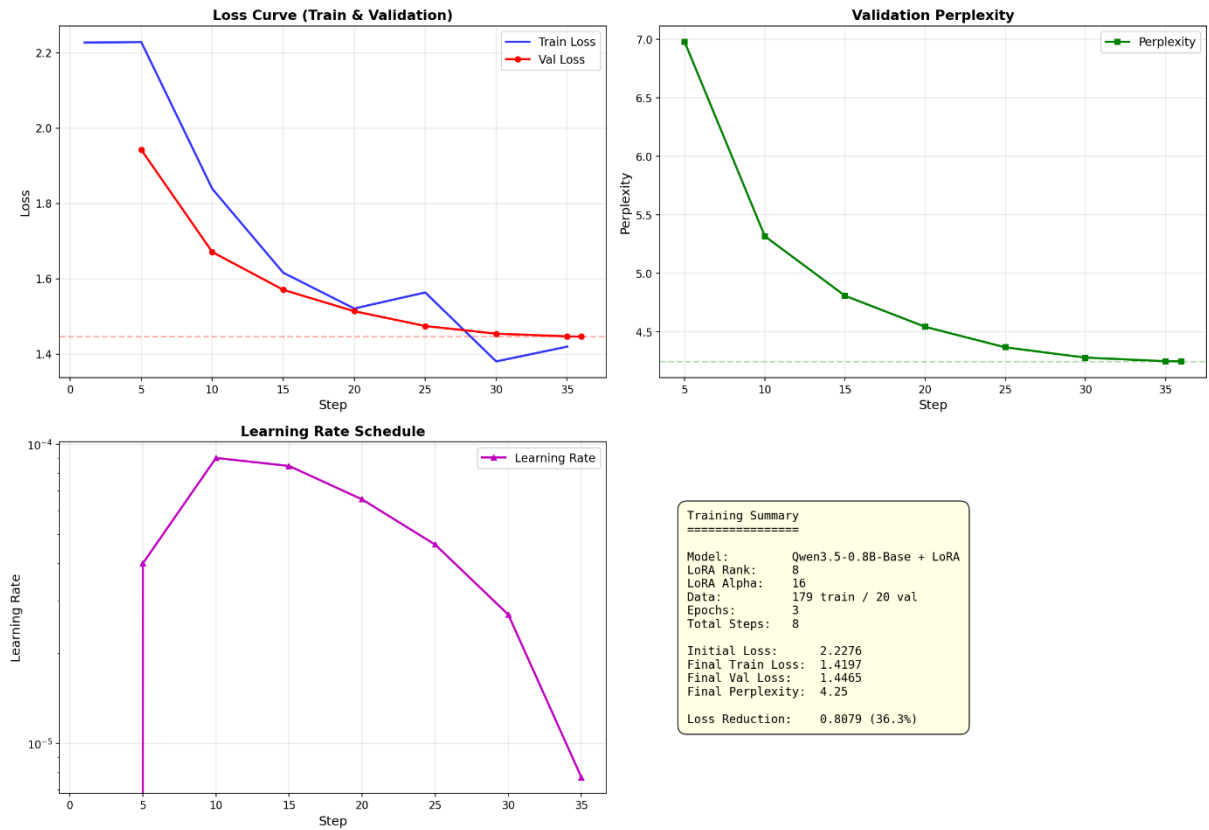
```
{"question": "五味阴阳之用如何区分? ", "answer": "辛甘发散为阳, 酸苦涌泄为阴, 咸味涌泄为阴, 淡味渗泄为阳。", "source": "黄帝内经_素问.md", "chunk_id": 43}
```

7.3 Training Configuration

Configuration	Value
Base Model	Qwen3.5-0.8B-Base (AutoModelForCausalLM)
Fine-tuning Method	LoRA (r=8, alpha=16, dropout=0.1)
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Training Epochs	10 epochs (actually completed ~3.6 epochs / 36 steps)
Batch Size	4, gradient accumulation: 4
Learning Rate	1e-4
Precision	fp16
Optimizer	AdamW

7.4 Experimental Results

Qwen3.5-0.8B-Base LoRA Fine-tuning - Training Metrics



Training took 20 minutes. Core metrics:

Metric	Initial	Final	Change
Train Loss	2.23	1.42	-36.3%
Val Loss	1.94	1.45	-25.3%
Perplexity	6.98	4.25	-39.1%

The gap between train and validation loss is small (1.42 vs 1.45), indicating no significant overfitting.

Comparative test results (original model vs. LoRA fine-tuned model):

Test Question	Original Model	LoRA Fine-tuned Model
---------------	----------------	-----------------------

Which acupoints for abdominal distension?	Zusanli. (33 chars)	Zusanli. (4 chars, concise)
Which acupoints for headache?	Hand-Shaoyang Sanjiao meridian. (7 chars)	Foot-Taiyang Bladder meridian. Fengchi acupoint... (514 chars, detailed but repetitive)
What are the twelve meridians?	Hand-Foot Yangming meridians... (34 chars)	Hand-Taiyin, Hand-Shaoyin, Hand-Jueyin. (15 chars)
What are the contraindications of acupuncture?	Needles should not be inserted too deeply... (31 chars)	1. Needles cannot enter yin divisions; (10 chars)
What are the effects of Zusanli?	Treats numbness, limb insensitivity. (8 chars)	Zusanli, primarily elevates stomach qi... (25 chars, more professional)
What is the function of acupoints?	Acupoints are the direct action sites for acupuncture treatment. (15 chars)	Acupoints are specific locations for treating diseases... (94 chars, more detailed)

7.5 Analysis

The LoRA + SFT approach in v4 achieved favorable results across multiple metrics: (1) Significant loss reduction with a small train/validation gap, indicating the model learned effective knowledge without overfitting; (2) Perplexity decreased to 4.25, demonstrating significantly improved predictive capability on TCM texts; (3) In comparative testing, the fine-tuned model provided more professional and detailed TCM responses on certain questions.

However, some responses still exhibited repetition issues (e.g., "Fengchi acupoint" repeated approximately 100 times in the headache question), indicating that under limited

training steps (36 steps), the model has not fully converged. It is recommended to expand the QA pairs to 500+, train for 5–8 epochs, and incorporate repetition penalty mechanisms.

8 Conclusion

The data volume across the four versions ranges from 166 to 1,438 samples. A noteworthy phenomenon is that v2 (175 training samples) exhibited severe overfitting, while v4 (161 training samples) performed well. The key difference lies in the fact that v2 used 30 epochs, whereas v4 used only approximately 3.6 epochs. This demonstrates that early stopping and appropriate regularization are crucial in small-data scenarios.

Furthermore, Catastrophic Forgetting is one of the most critical issues in continual pre-training. The v3 experiment provides a vivid case study: while the model learned to continue TCM text, it severely degraded its original general QA capabilities. The root cause of this phenomenon lies in the "Stability-Plasticity Dilemma" of neural networks: the network must adjust weights when learning new knowledge, but these adjustments may overwrite existing important knowledge. Moreover, pre-training is time-consuming; this experiment prepared data from only 6 books but consumed 4 hours of training time. For small and medium-sized enterprises with limited resources, the cost-effectiveness of pre-training is inferior to waiting for the latest open-source models.

Comparing all four versions, the RAG approach is the most portable and least resource-intensive solution. In the small-sample setting of this experiment, it also achieved the best enhancement effect on QA tasks.

9 Future Work

Data expansion: Extend TCM QA pairs to 500–1,000, covering broader TCM sub-domains (formulas, diagnostics, health preservation, etc.).

Hybrid training: Experiment with a two-stage CPT + SFT training pipeline to quantitatively analyze the actual extent of catastrophic forgetting.

Multi-dimensional evaluation: In addition to loss and perplexity, introduce generation quality metrics such as ROUGE, BLEU, and BERTScore.

RAG integration: Combine the fine-tuned embedding model (v1 approach) with the generative model (v4 approach) to build a complete RAG system.

Model quantization and deployment: Use GPTQ/AWQ to quantize models for efficient inference on edge devices.

References

- [1] Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
- [2] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
- [3] BAAI. BGE-M3: Multi-Lingual, Multi-Granularity, Multi-Epoch Embedding Model.
<https://github.com/FlagOpen/FlagEmbedding>.
- [4] Qwen Team. (2025). Qwen3.5 Technical Report. Alibaba Group.
- [5] McCloskey, M., & Cohen, N. J. (1989). Catastrophic Interference in Connectionist Networks. The Psychology of Learning and Motivation.
- [6] Kirkpatrick, J., et al. (2017). Overcoming catastrophic forgetting in neural networks. PNAS.
- [7] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- [8] Project code and experimental data: v1–v4 directories,
<https://github.com/EthanGD/NLPHomeWork20260530>.